

RPNet 概述

功能 从 SAC / MSEED 波形自动判定 P 波初动极性：U（向上）、D（向下）、K（不确定）。

环节	处理内容	输出
到时整理（可选）	TauP 理论到时替换或补齐 P/S	统一到时表
极性预测	逐台迭代预测极性与置信度	pol_result.csv
结果筛选	不稳的判为 K 并剔除	筛后极性表
SKHASH 输入	生成 HASH 格式输入与控制文件	hash2 或 hash3 目录

两级流程：RPNet 出极性，SKHASH（Skoumal et al., 2024）反演震源机制。

本地运行版本：RPNet v0.1.0（Han et al., 2025, SRL）。

处理流程

模块

输入与输出

- 0. 数据准备 事件目录、震相、台站三个 CSV + 事件波形目录
- 1. 到时整理（可 TauP 理论到时； add_sta 给无拾取台站补到时
选）
- 2. 极性预测 垂直分量切窗（ $P \pm 2.5$ s、高通 1 Hz、100 Hz） $\longrightarrow$
RPNet_v1.h5
- 3. 筛选与输出 阈值筛选 $\longrightarrow$ pol_result.csv 与 SKHASH 输入文件
- 4. 震源机制 SKHASH 网格搜索 $\longrightarrow$ 走向、倾角、滑动角

模型权重随仓库附带（model/RPNet_v1.h5，约 62 MB）；v0.1.0 没有训练接口，作者预告后续版本。

模块 1–3 由一个入口 run_RPNet.py 一次完成；模块 4 单独运行 SKHASH 命令。

按目标选择流程

只要极性 跑 `run_RPNet.py`, 取 `output01/pol_result.csv`, 跳过 SKHASH。

极性 + 震源机制 example 流程（本教程主线），再跑一步 SKHASH。

极性 + S/P 振幅比 example2 流程, `hash_version='hash3'`, 需要三分量波形。

训练本区模型 v0.1.0 未提供；作者预告训练与微调模块。

四个输入文件齐备即可开始。两个示例的事件、波形、台站表全部随仓库附带。

程序目录

目录	用途
<code>example/</code>	熊本地震两个事件（hash2，仅极性）+ 输入与参考输出
<code>example2/</code>	2024 Buan 两个事件（hash3，极性 + S/P 比）+ 输入与参考输出
<code>model/</code>	RPNet_v1.h5 与 mRPNet_v1.h5（各约 62 MB，示例用前者）
<code>src/rpnet/</code>	<code>predict.py</code> （到时、切窗、预测）与 <code>rpnet2skhash.py</code> （输出转换）
<code>dist/</code>	<code>rpnet-0.1.0</code> 的 wheel 与源码包（离线安装用）

两个示例目录各带 `run_RPNet.py`、`hyperparams.py`、`control_file0.txt`，并附分步教程 `run_RPNet.ipynb`。

源码与工作目录

从 github.com/jongwon-han/RPNet 克隆或下载源码包并解压。

保留原件，将程序目录复制为 `~/RPNet-work`，并在终端核对内容：

```
export RPNET_ROOT="$HOME/RPNet-work"
cd "$RPNET_ROOT"
ls README.md model example example2
```

示例中的模型路径写作 `../model/RPNet_v1.h5`，相对 `example` 目录——在示例目录内运行就不用改脚本。

新终端需重新设置 `RPNET_ROOT`。

运行环境：Conda

安装 Miniconda 或 Anaconda 后，创建环境并安装：

```
conda create -n rpnet python=3.9 -y
```

```
conda activate rpnet
```

```
pip install rpnet
```

```
pip install skhash
```

```
python -c "import rpnet, SKHASH; print('import OK')"
```

pip install rpnet 按作者清单装齐：tensorflow 2.7.0、numpy 1.19.5、obspy 1.3.1、

pandas 1.4.4、keras-self-attention、protobuf 3.20.0 等，下载量约 0.5 GB。

skhash (1.1.5) 依赖 numpy<2.0，与上面兼容，装进同一环境即可。

默认 CPU 运行 (gpu_num=""); 示例数据小，作者注明 CPU 更快。

GPU 需为 TensorFlow 2.7 另配 CUDA 11.x 与 cuDNN 8.1，本教程未验证。

网格步长：本教程改用 5 度

模板默认 `dang 1`（1 度网格）必须配编译好的 Fortran 例程：

纯 Python 回退要分配约 34 GiB 内存，且并行模式下报错被吞、`out.csv` 只剩表头。

本教程在 `control_file0.txt` 改两处，用 SKHASH 内置的 Python 例程，免编译（1 度路线见下一页）：

```
$dang          # minimum grid spacing (degrees)
5
$use_fortran    # Fortran subroutine for fast grid search
False
```

实测（5 度、纯 Python）：example1 全程 2.5 秒、内存峰值约 1 GB；example2 约 24 秒。代价：机制解与作者的 1 度结果相差约 1–5 度（见结果核对页）。

可选：1 度网格与 Fortran 例程

要完全复现作者设置（dang 1）按本页编译；5 度主线不需要本页。**Ubuntu / WSL2：先装编译器，再进入 SKHASH 的 functions 目录**

```
sudo apt-get install -y gfortran
```

```
cd ~/anaconda3/envs/rpnet/lib/python3.9/site-packages/SKHASH/functions
```

把 gridsearch.f 中 ncoor=31032 改为 ncoor=4000000（1 度网格需 3,745,620；5 度用原值即可），再编译：

```
SETUPTOOLS_USE_DISTUTILS=stdlib \
```

```
python -m numpy.f2py -c gridsearch.f -m gridsearch
```

成功标志：生成 gridsearch.cpython-39-*.so、SKHASH 打印 Fortran loaded；并把 control_file0.txt 的 use_fortran 改回 True。

需本环境的 Python 3.9 + numpy 1.19.5；更高版本 numpy.distutils 已不可用。

输入数据：四个文件

文件	示例	内容
事件目录	Kumamoto_catalog.csv	data_id、jst（发震时刻）、lat、lon、dep、mag
震相表	Kumamoto_phase.csv	event_id、data_id、sta、ptime、stime、pol
台站表	hinet_station.csv	net、sta、chan、lat、lon、elv
波形目录	waveform/<data_id>/<sta>.*	如 AKAIKE.U.SAC；垂直通道 *Z 优先，无则 *U

列名可换：ftime / fwfid / fptime / fstime 在 hyperparams.py 里指向实际列名。

事件目录与震相的时间须同一时间标准（jst 只是列名，不做换算）；

非 100 Hz 的波形会自动插值到 100 Hz。

hyperparams.py: 路径与字段 (example)

修改 `example/hyperparams.py` 开头的参数段:

```
model='../model/RPNet_v1.h5'  
wf_dir='../waveform'  
event_catalog='../Kumamoto_catalog.csv'  
phase_metadata='../Kumamoto_phase.csv'  
sta_metadata='../hinet_station.csv'  
out_dir='../output01'  
ctrl0='../control_file0.txt'  
ftime='jst'; fwfid='data_id'; fptime='ptime'; fstime='stime'
```

路径相对 `example` 目录; 换自己数据时改这五个输入路径。

`ctrl0` 是 SKHASH 模板, 运行前先把速度模型路径改成实际位置 (见后页)。

`example2` 同名文件结构相同, 默认值即其示例所需。

hyperparams.py: 预测与筛选参数

```
cores=5; batch_size=2**13; iteration=100; gpu_num=""  
std_threshold=0.2; mean_threshold=0; rm_unknwon=True  
change2taup=True; add_sta=True; keep_initial_phase=False  
taup_model='iasp91'; hash_version='hash2'
```

iteration=100 迭代预测: prob 取均值、std 取标准差; 0 为单次确定性输出

std_threshold=0.2 std 超过 0.2 的判为 K

rm_unknwon=True 生成 SKHASH 输入前剔除 K

change2taup=True 用 TauP 理论到时替换到时, 原值存 ptime0/stime0

add_sta=True 给有波形、无拾取的台站补理论到时; 必须配合

change2taup=True

keep_initial_phase 已有拾取时保留原值 (example2 用 True)

SKHASH 模板 control_file0.txt

除前页两处外，必改第三处——速度模型路径：

```
$vmodel_paths
/home/your/RPNet-work/example/vz.iasp91
```

参数	本例值	含义
npolmin	8	少于 8 条极性的事件不反演
dang	5	网格角步长（度）；模板原值 1
delmax	120	震中距上限，超出极性被丢弃
	km	
num_cpus	2	并行核数
use_fortran	False	内置 Python 例程；模板原值 True
example 自带 vz.iasp91（深度 km、Vp 两列）；example2 从 example 拷一份，换区时用本区模型。		

运行 example1

```
conda activate rpnet  
cd "$RPNET_ROOT/example"  
python run_RPNet.py
```

屏幕依次显示：TauP (P) $\rightarrow$ TauP (S) $\rightarrow$ Make input matrix $\rightarrow$ Predict polarity $\rightarrow$ ALL DONE。

实测（WSL2、5 核、CPU）：全程约 0.3 分钟。

脚本开头会删除并重建 output01/——需要保留的结果先复制到别处。

run_RPNet.ipynb 是同一流程的分步版本，便于逐段查看中间表。

RPNet 输出文件 (output01/)

文件或子目录

内容

pol_result.csv

全部预测: predict (U/D/K)、prob、std 与 TauP 前后到时

MSEED/<id>/<sta>.mseed

切窗存档 ($P \pm 2.5$ s、高通 1 Hz、100 Hz)

hash2/uniq_pol.csv

筛选 (去 K、去重台站) 后的极性表

hash2/IN/station.txt、

HASH 格式台站与震相文件

phase.txt

hash2/control_file.txt

路径已指向 output01, 可直接交给 SKHASH

example1 特有一步: 台站统一改名 S001...S0xx、net=HI、chan=HHZ, 是 Hi-net 的整理 (Renaming 段, 仅为格式一致), 换台网时按需改写。

运行 SKHASH 与结果核对

在 example 目录内运行（控制文件里是相对路径）：

`SKHASH output01/hash2/control_file.txt`

实测（5 度、纯 Python）2.5 秒；屏幕先提示丢弃震中距超过 120 km 的极性。

结果在 `output01/hash2/OUT/`： `out.csv`（strike、dip、rake、quality）、`out2.csv` 与 `figure/`。

事件	本次（5 度）	作者随包（1 度）	目录参考
M4.4	265.4 / 41.4 / -88.2	265.0 / 39.8 / -89.2	267 / 35 / -91
M4.0	22.7 / 73.0 / -179.2	22.9 / 72.2 / -178.6	20 / 69 / 172

5 度与作者 1 度结果相差 1–2 度；目录参考是另一套机构解，量级相符。

example2: S/P 振幅比 (hash3)

数据: 2024 年韩国 Buan M4.8 与 M3.1, 三分量波形 (HGE / HGN / HGZ)。

与 example1 的 hyperparams.py 差异:

```
hash_version='hash3'; std_threshold=0.02; mean_threshold=0.95
```

```
keep_initial_phase=True; fwfid='pick'; ftime='utc'
```

```
sp_freq=(1,20)
```

```
sp_win=[-2.5,-0.5,-0.5,+1.0,-0.5,+2.5]
```

sp_win 六个数依次为噪声窗、P 窗 (相对 P 到时)、S 窗 (相对 S 到时) 的起止秒; S/P 比写入 hash3/IN/amp.txt。运行前同样先改 control_file0.txt 的 vmodel_paths (文本模型从 example 拷入)。

实测 (5 度): RPNet 约 1.2 分钟, SKHASH 约 24 秒; 机制解与作者 1 度随包相差几度

(M4.8 事件本次 33.6/86.2/173.3, 作者随包 33.7/87.9/174.8)。

换成本区数据

项目	做法
三个 CSV	保留同名列即可；列名不同就改 <code>ftime/fwfid/fptime/fstime</code>
波形目录	按 <code>wf_dir/<fwfid>/<sta>.*</code> 组织，垂直通道 *Z 优先
台站改名段	example1 的 Renaming 段是 Hi-net 专用；自有台网保留原名或改短名（HASH 震相行台站列宽有限，建议不超过 4 字符）
速度模型	<code>vmodel_paths</code> 换本区一维模型：深度 km、Vp 两列文本
TauP 模型	<code>taup_model</code> 可换 'ak135'，或填自定 npz 绝对路径（example2 附 <code>srkim_iasp.npz</code> 示例）
时间标准	目录与震相同一标准即可，用 UTC 最省事

注意事项与已知坑

输出目录被清空 每次运行先 `rmtree(out_dir)`，旧结果不留底。

example1 的笔误 `mean_threshold` 非 0 时会 `NameError`（脚本里拼成 `mean_thresuld`）；要启用就照 `example2` 的写法改这一行。

TauP 无容错 某台站一事件对算不出理论到时会直接报错停住，先从表里清掉这类组合。

并行吞错 SKHASH 的 `num_cpus>1` 不显示 worker 报错；`out.csv` 为空时改回单核重跑，看真实错误。

结果涨落 迭代预测带随机性：极性表可能差一两条，机制解在小数位上浮动。

1 度路线 要走作者的 `dang 1` 需按可选页编译 Fortran 并扩 `ncoor`；未编译而设 `use_fortran=True` 会交互提问。

遇到错误时先查什么

现象

导入失败

Can not find model
file

out.csv 只有表头

分配 34 GiB 失败

rotations too big

TauP 阶段停住

极性几乎全 K

处理办法

激活 rpnet 环境；确认 rpnet、skhash 装在同一环境

model 路径相对 example 目录；从别处运行要改成绝对
路径

num_cpus 改 1 重跑看报错（并行不显示 worker 错误）

1 度网格走了 Python 回退：改 dang 5，或编译 Fortran
例程

1 度路线的 ncoor 容量不足：改大后重新编译

清理算不出理论到时的台站—事件对

std_threshold 太严或波形质量差；先看 pol_result.csv
的 prob/std 分布

每一步的结果交给哪个程序

步骤	输出	后续
数据准备	目录 + 震相 + 台站 CSV + 波形 目录	run_RPNet.py
到时整理	TauP 到时表 (ptime/stime)	切窗与预测
极性预测	pol_result.csv (含 prob/std)	人工核对 / SKHASH
筛选与转换	hash2 或 hash3 输入 + 控制文件	SKHASH
SKHASH	OUT/out.csv (走向、倾角、滑动 角)	结果分析

先把两个示例完整跑通，再把同样的改动套到本区数据上。